

Report Tecnico: Esplorazione e Analisi di Windows PowerShell

Studente: Bejte Gerald

Istituzione: Epicode

Ambito: Cybersecurity & System Administration

Introduzione

Il presente documento illustra l'attività di laboratorio focalizzata sull'utilizzo di **Windows PowerShell**, un framework di gestione della configurazione e automazione delle attività. Come studente di Cybersecurity presso Epicode, l'analisi di questo strumento è fondamentale: PowerShell non è solo una shell interattiva, ma un potente motore basato su .NET Core (o .NET Framework nelle versioni legacy) che permette un controllo granulare del sistema operativo.

Immaginiamo PowerShell non come una semplice finestra in cui digitare comandi testuali, ma come un'interfaccia diretta che ti permette di mettere le mani sugli ingranaggi interni del computer. Dire che è un motore basato su .NET Core o .NET Framework significa che ogni singola azione che compi attraverso la shell non è un comando isolato, ma una chiamata a una vastissima libreria di funzioni prefabbricate da Microsoft che gestiscono la memoria, la sicurezza e le comunicazioni del sistema.

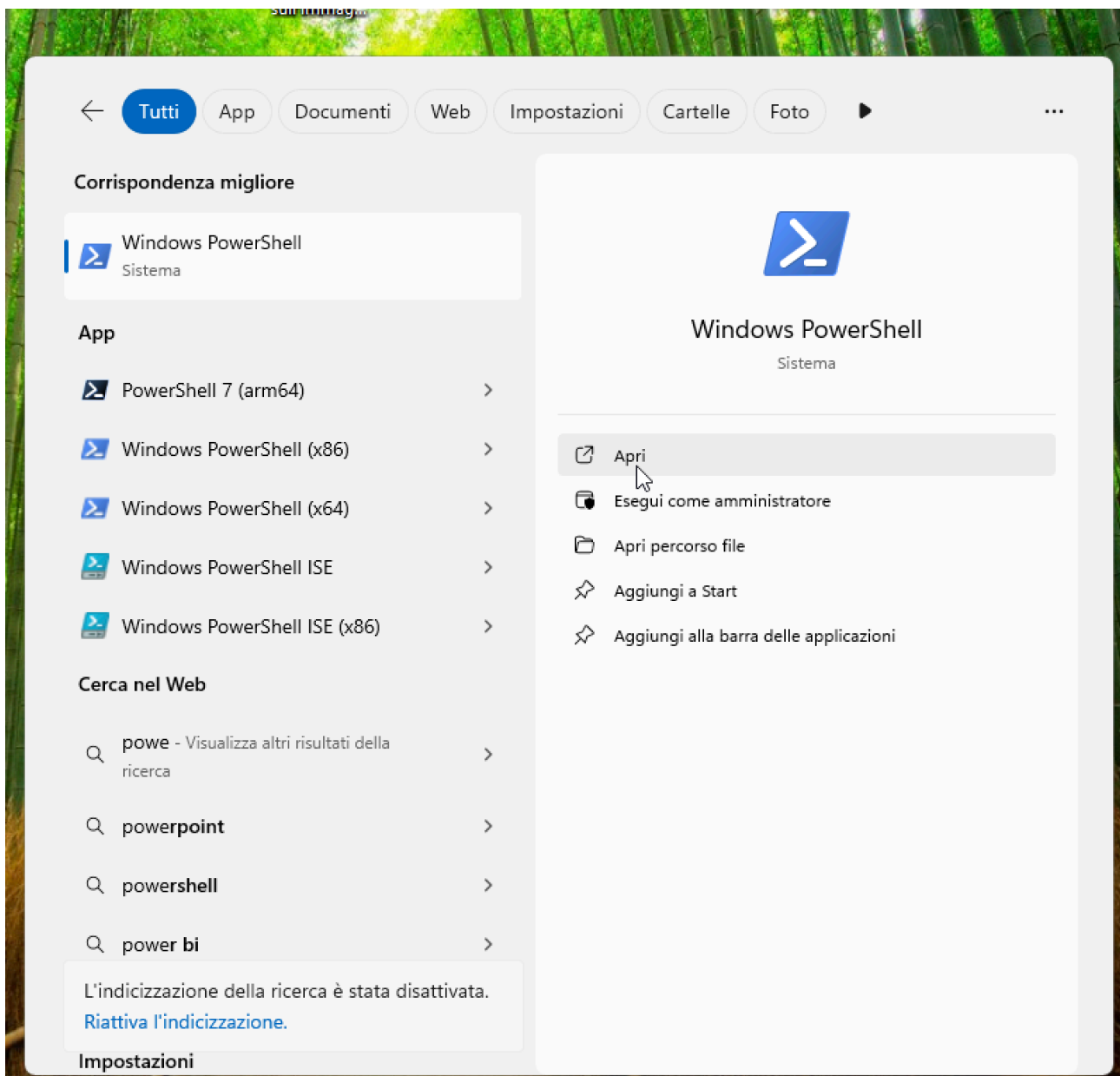
In una shell tradizionale, quando chiedi una lista di file, ricevi solo del testo. In PowerShell, grazie al supporto di .NET, ricevi un oggetto "vivo" che contiene tutte le proprietà di quel file, come la data di creazione o i permessi di accesso, permettendoti di manipolarlo con una precisione chirurgica. Nelle versioni legacy legate al .NET Framework, questo potere era confinato esclusivamente all'ambiente Windows, mentre il passaggio a .NET Core ha trasformato PowerShell in uno strumento universale capace di girare e gestire allo stesso modo macchine Linux e macOS.

Per chi si occupa di sicurezza, questa natura "orientata agli oggetti" significa poter interrogare il sistema operativo non per leggere semplici stringhe, ma per analizzare in profondità i processi attivi, le connessioni di rete e i registri di sistema come se si stesse programmando direttamente il cuore della macchina, rendendo l'automazione delle difese o l'analisi delle vulnerabilità estremamente più fluida e strutturata rispetto ai sistemi del passato.

In un'ottica di **Risk Management** e **Internal Audit**, comprendere PowerShell è cruciale per il monitoraggio delle attività di rete, la gestione dei permessi e l'automazione dei processi di difesa. Il laboratorio si articola in cinque fasi distinte, che spaziano dall'interazione base con la console fino alla gestione dei processi di sistema e di rete.

Parte 1: Accesso alla Console PowerShell

L'accesso alla console rappresenta il primo step operativo. In ambiente Windows, PowerShell può essere eseguito con privilegi utente standard o come **Amministratore**. Per scopi di analisi forense o configurazione di sistema, l'esecuzione con privilegi elevati è spesso necessaria per accedere a porzioni protette del kernel e del file system.



Parte 2: Esplorazione dei Comandi (Prompt vs PowerShell)

PowerShell offre la retrocompatibilità con i classici comandi del Prompt dei Comandi (CMD) attraverso l'uso di **alias**.

- **Comandi testati:** `dir`, `cd`, `cls`.
- **Analisi:** Sebbene digitando `dir` l'output sembri identico al CMD, in PowerShell `dir` è un alias per il cmdlet `Get-ChildItem`. Questo significa che l'output non è semplice testo, ma una collezione di **oggetti** dotati di proprietà e metodi.

Una shell tradizionale, come Bash su Linux o il Prompt dei comandi (cmd.exe) su Windows, funziona fondamentalmente come un interprete di puro testo. Immaginala come una conversazione scritta: tu digiti un comando, il sistema lo esegue e ti restituisce il risultato sotto forma di stringhe di caratteri. Se chiedi al sistema l'elenco dei file in una cartella, una shell tradizionale ti mostra semplicemente i nomi dei file, le dimensioni e le date come se fossero scritte su un foglio di carta digitale.

Il limite principale di questo approccio "text-based" emerge quando devi elaborare quei dati. Se volessi isolare solo i file più grandi di un gigabyte creati nell'ultima settimana, la shell tradizionale non "capisce" cos'è una data o cos'è una dimensione; vede solo testo. Per ottenere il risultato, dovresti usare altri strumenti esterni per tagliare, filtrare e manipolare quelle stringhe di testo fino a estrarre l'informazione che ti serve. È un processo che richiede spesso incastri complessi perché il sistema non passa informazioni strutturate, ma solo flussi di parole e numeri che devono essere interpretati manualmente dall'utente o da altri piccoli programmi messi in serie.

Il Prompt dei comandi (cmd.exe) è l'erede diretto di MS-DOS, un sistema nato negli anni '80 quando l'informatica era molto più semplice. La differenza fondamentale con PowerShell non sta solo nella "potenza", ma nella **natura logica** di ciò che gestiscono: il Prompt manipola **testo**, mentre PowerShell manipola **oggetti**.

Nel Prompt dei comandi, ogni output è una semplice sequenza di caratteri stampata a video. Se chiedi la lista dei processi attivi, il Prompt ti restituisce una tabella di testo "morta": per il sistema sono solo lettere e numeri messi in fila. Se volessi estrarre solo i processi che consumano molta memoria, dovresti scrivere script complessi per "ritagliare" il testo colonna per colonna, sperando che l'allineamento dei caratteri non cambi mai, altrimenti lo script si rompe. È uno strumento rigido, pensato per eseguire file eseguibili esterni (.exe) o semplici file batch (.bat), ma con pochissima intelligenza interna.

PowerShell, invece, è un ambiente di programmazione completo che "capisce" cosa sta visualizzando. Quando chiedi la stessa lista di processi in PowerShell, non ricevi del testo, ma una collezione di oggetti digitali provenienti dal framework .NET. Ogni processo nella lista è un'entità che ha già in sé le proprietà "Nome", "ID" e "Memoria". Questo significa che puoi chiedere a PowerShell di "filtrare tutti i processi dove la memoria è superiore a 100MB" con un comando semplicissimo, perché la shell sa cos'è la memoria e sa come confrontare i numeri, senza dover interpretare stringhe di testo. Mentre il Prompt è un maggiordomo che sa solo eseguire ordini semplici e leggere etichette, PowerShell è un ingegnere che conosce la struttura interna di tutto ciò che tocca.

Caratteristica	Prompt dei Comandi (CMD)	PowerShell
Output	Testo (Stringhe)	Oggetti .NET
Logica	Semplice esecuzione comandi	Linguaggio di scripting completo
Piattaforme	Solo Windows	Windows, Linux, macOS
Estensibilità	Limitata	Altissima tramite Moduli

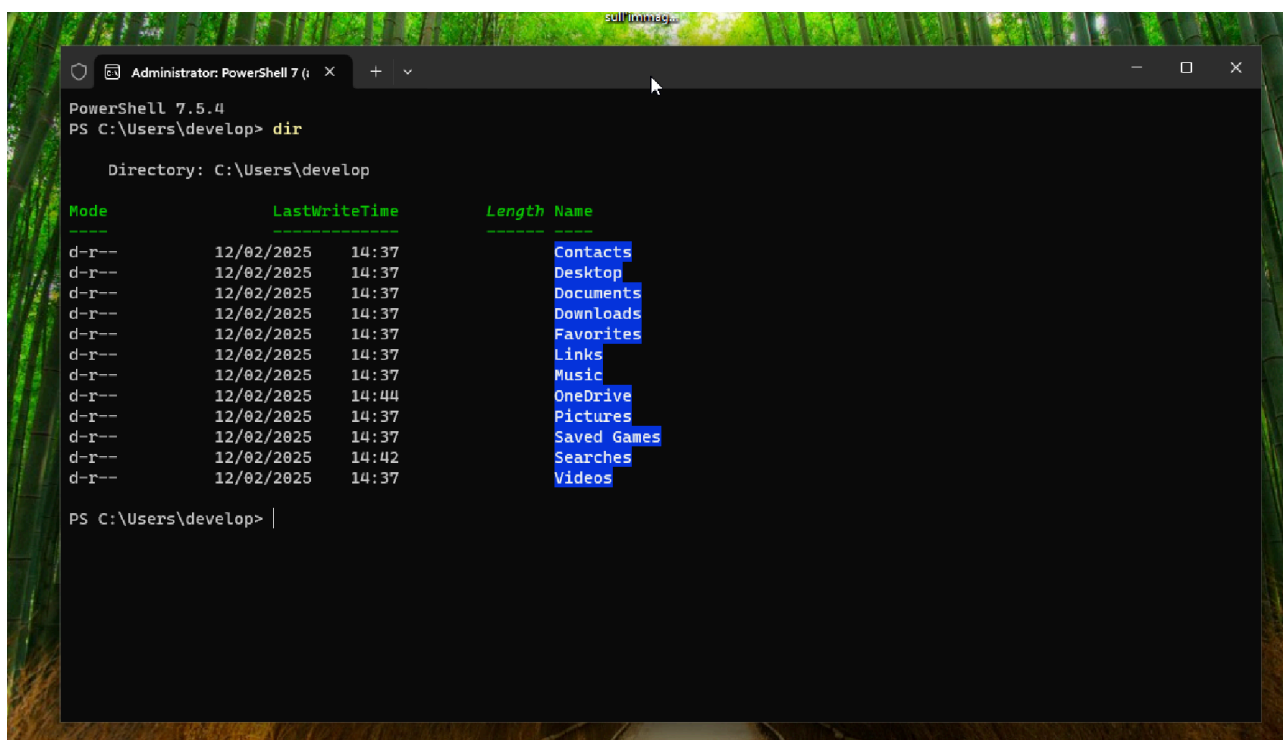
Il comando **dir** in PowerShell

In PowerShell, **dir** non è un comando reale, ma un **alias** (un soprannome) del cmdlet **Get-ChildItem**. Quando lo digiti, PowerShell non sta restituendo del testo, ma sta creando una collezione di **oggetti .NET** di tipo **System.IO.FileInfo** o **System.DirectoryInfo**.

La tabella a video è solo una "fotografia" parziale di quegli oggetti, formattata per essere leggibile. In realtà, ogni riga della lista possiede decine di proprietà nascoste (permessi, hash, attributi di sicurezza, metodi per rinominarli o spostarli) che puoi richiamare istantaneamente. Poiché PowerShell lavora con oggetti e non con testo, puoi filtrare i file per dimensione o data di creazione usando operatori logici matematici (come "maggiore di" o "minore di"), perché la shell "sa" che la dimensione è un numero e la data è un momento preciso nel tempo, non semplici parole scritte a schermo.

l'output mostra diverse colonne:

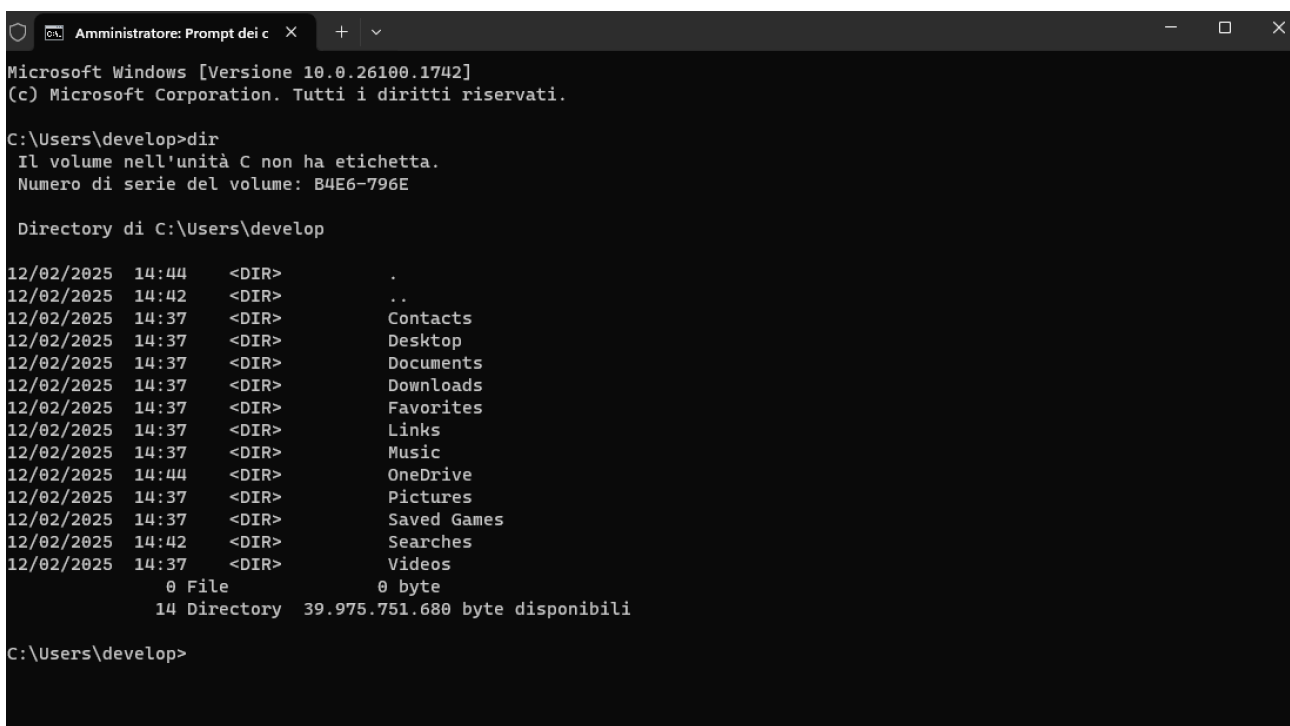
- **Mode:** **d-r--** significa che è una **Directory** (cartella) ed è **Read-only** (sola lettura) per certi aspetti di sistema.
- **LastWriteTime:** L'ultima volta che hai toccato quella cartella.
- **Name:** Il nome del collegamento (Contacts, Desktop, ecc.).



Il comando **dir** nel Prompt dei comandi (CMD)

Nel Prompt dei comandi, **dir** è un comando interno del sistema operativo che risale all'epoca del DOS. Quando lo esegui, il computer interroga il file system e genera una **rappresentazione testuale** di ciò che trova.

Il risultato che vedi a schermo è una stringa di caratteri piatta. Se volessi usare il risultato di **dir** in un altro comando (magari per eliminare solo i file creati in una certa data), dovresti trattare quell'output come un blocco di testo, tagliando e cucendo le righe con comandi complessi e fragili. Se cambi la lingua del sistema da italiano a inglese, ad esempio, la parola "Directory" cambia e il tuo script potrebbe rompersi perché non trova più la corrispondenza testuale.



```
Microsoft Windows [Versione 10.0.26100.1742]
(c) Microsoft Corporation. Tutti i diritti riservati.

C:\Users\develop>dir
Il volume nell'unità C non ha etichetta.
Numero di serie del volume: B4E6-796E

Directory di C:\Users\develop

12/02/2025  14:44    <DIR>        .
12/02/2025  14:42    <DIR>        ..
12/02/2025  14:37    <DIR>        Contacts
12/02/2025  14:37    <DIR>        Desktop
12/02/2025  14:37    <DIR>        Documents
12/02/2025  14:37    <DIR>        Downloads
12/02/2025  14:37    <DIR>        Favorites
12/02/2025  14:37    <DIR>        Links
12/02/2025  14:37    <DIR>        Music
12/02/2025  14:44    <DIR>        OneDrive
12/02/2025  14:37    <DIR>        Pictures
12/02/2025  14:37    <DIR>        Saved Games
12/02/2025  14:42    <DIR>        Searches
12/02/2025  14:37    <DIR>        Videos
               0 File             0 byte
              14 Directory  39.975.751.680 byte disponibili

C:\Users\develop>
```

- **La Struttura:** L'output è discorsivo. Ti dice "Il volume nell'unità C non ha etichetta" e aggiunge il riepilogo finale con il conteggio dei byte disponibili. Queste informazioni sono utili da leggere, ma sono un incubo da automatizzare perché sono **puro testo**.
- **Il Dettaglio Tecnico:** Vedi l'indicazione <DIR>. Per il Prompt, quella è solo una stringa di caratteri stampata tra la data e il nome. Se volessi estrarre solo i nomi delle cartelle per un report di cybersecurity, dovresti dire al computer: "Prendi la riga, conta quanti spazi ci sono, e se trovi la scritta <DIR> allora prendi quello che viene dopo". Se cambia la lingua del sistema, il tuo script si rompe.
- **Rigidità:** Noterai anche i riferimenti . e .. (la cartella corrente e quella superiore). Il Prompt li mostra sempre come parte del testo del file system, mentre PowerShell spesso li nasconde per offrirti una visuale più pulita degli oggetti reali

Il comando ipconfig in PowerShell 7.5.4 (Core)

In questa versione ultra-moderna, **ipconfig** è tecnicamente un "processo figlio" che PowerShell 7 ospita. Mentre il comando ti restituisce i dati familiari, il contesto è cambiato radicalmente rispetto alle versioni legacy: ora tutto gira su **.NET Core (CoreCLR)**. Questo significa che PowerShell non sta semplicemente leggendo i dati di Windows, ma sta gestendo l'output attraverso un runtime cross-platform progettato per alte prestazioni.

Se utilizzi il comando nativo **Get-NetIPAddress**, PowerShell 7.5.4 trasforma le informazioni della tua scheda di rete in **oggetti .NET ad alta efficienza**. Per un analista di cybersecurity, questo è un vantaggio enorme: grazie al core aggiornato, la velocità con cui puoi filtrare migliaia di parametri di rete o analizzare pacchetti è superiore, poiché il sistema non deve tradurre continuamente dati tra vecchie architetture e il nuovo hardware M1.

Analisi tecnica dell'output su PS 7.5.4:

- **Indirizzo IPv4 (192.168.64.23):** In questa versione, l'IP non è solo una stringa, ma può essere manipolato come un oggetto **IPAddress**, permettendoti di calcolare istantaneamente maschere di rete o range di scansione per attività di penetration testing.
- **Efficienza Cross-Platform:** Essendo su .NET Core, la logica con cui PowerShell gestisce questo output è la stessa che useresti su Kali Linux

```
PS C:\Users\develop> ipconfig

Configurazione IP di Windows

Scheda Ethernet Ethernet:

    Suffisso DNS specifico per connessione: nexxt
    Indirizzo IPv6 . . . . . : fda7:9880:7399:3122:d243:1039:bcf0:1d28
    Indirizzo IPv6 temporaneo. . . . . : fda7:9880:7399:3122:c4bd:5b49:c1b:80ae
    Indirizzo IPv6 locale rispetto al collegamento . : fe80::d0f3:5456:4607:a6c4%5
    Indirizzo IPv4. . . . . : 192.168.64.23
    Subnet mask . . . . . : 255.255.255.0
    Gateway predefinito . . . . . : fe80::10bd:3aff:fea6:64%5
                                   192.168.64.1

PS C:\Users\develop> |
```

Il comando ipconfig nel Prompt dei comandi (Legacy)

Il Prompt dei comandi (CMD) rimane invece ancorato a una logica degli anni '90. Nonostante giri su un sistema moderno, l'eseguibile **ipconfig.exe** all'interno del CMD produce un **output testuale statico e isolato**. Non c'è un framework sottostante che permetta al CMD di "capire" i dati; si limita a proiettare pixel a forma di numeri sullo schermo.

E' fondamentale evidenziare che il CMD non beneficia delle ottimizzazioni del runtime .NET 9. Se dovessi automatizzare la raccolta degli IP da cento macchine diverse, usare il CMD ti costringerebbe a creare complessi parser di testo che fallirebbero al minimo cambio di formattazione del sistema operativo. Al contrario, la tua PowerShell 7.5.4 tratta questi dati come **informazioni strutturate**, rendendo l'attività di monitoraggio della sicurezza infinitamente più solida e professionale.

```
C:\Users\develop>ipconfig

Configurazione IP di Windows

Scheda Ethernet Ethernet:

    Suffisso DNS specifico per connessione: nexxt
    Indirizzo IPv6 . . . . . : fda7:9880:7399:3122:d243:1039:bcf0:1d28
    Indirizzo IPv6 temporaneo. . . . . : fda7:9880:7399:3122:c4bd:5b49:c1b:80ae
    Indirizzo IPv6 locale rispetto al collegamento . : fe80::d0f3:5456:4607:a6c4%5
    Indirizzo IPv4. . . . . : 192.168.64.23
    Subnet mask . . . . . : 255.255.255.0
    Gateway predefinito . . . . . : fe80::10bd:3aff:fea6:64%5
                                   192.168.64.1

C:\Users\develop>
```

Analisi degli Screenshot

Guardando le immagini, si può notare che:

- **Nel Prompt:** L'output è verboso e rigido. Se la lingua del sistema cambiasse, l'intero blocco di testo cambierebbe, rompendo eventuali automatismi.
- **In PowerShell 7:** Anche se il risultato sembra uguale, la shell evidenzia i comandi e i valori in modo diverso (sintassi colorata) perché sta già analizzando la struttura logica di ciò che ti sta mostrando. È un sistema che "sa cosa sta leggendo" mentre lo scrive.

Il comando `cd` in PowerShell

L'esecuzione del comando `cd` documentata nello screenshot permette di analizzare la transizione operativa dalla directory utente `C:\Users\develop` alla radice del sistema `C:\`. In PowerShell, questo spostamento non è una semplice variazione testuale ma un'interazione diretta con il cmdlet `Set-Location`. Per un analista impegnato nel Risk Management, la capacità di muoversi agilmente tra i diversi livelli gerarchici è il presupposto fondamentale per mappare la superficie di attacco e verificare l'integrità delle directory critiche.

L'ispezione condotta tramite il comando `dir` subito dopo il cambio di directory espone metadati essenziali che meritano un'analisi approfondita:

- **Attributi di protezione (Mode):** La colonna Mode mostra i permessi effettivi sulle cartelle radice. La presenza del flag `r` (Read-only) su directory come `Program Files` e `Users` conferma l'applicazione di policy di sistema volte a proteggere l'integrità dei binari e dei profili utente contro modifiche non autorizzate.
- **Analisi temporale (LastWriteTime):** La rilevazione di modifiche recenti su cartelle di sistema, come quella visibile per la directory `Windows` in data 20/02/2026, rappresenta un punto di controllo critico per l'Internal Audit. Ogni variazione in questi percorsi deve essere verificata per escludere la persistenza di minacce o alterazioni della configurazione di sicurezza.
- **Target sensibili:** L'elenco evidenzia directory come `PerfLogs`, spesso sottovalutate ma cruciali in ambito forense per il recupero di log sulle prestazioni che potrebbero nascondere tracce di esecuzioni malevole o anomalie di sistema.

Questa metodologia di esplorazione dimostra come PowerShell trasformi la semplice navigazione in un'attività di monitoraggio attivo. La comprensione dei percorsi assoluti e relativi, unita all'analisi dei metadati degli oggetti, fornisce una visibilità immediata sullo stato di conformità del sistema operativo, permettendo di isolare rapidamente aree sospette durante una sessione di Incident Response.

```
PS C:\Users\develop> pwd

Path
----
C:\Users\develop

PS C:\Users\develop> cd Documents
PS C:\Users\develop\Documents> cd \
PS C:\> dir

Directory: C:\

Mode                LastWriteTime         Length Name
----                -
d-----          01/04/2024    10:45          PerfLogs
d-r--          20/02/2026    12:08      Program Files
d-r--          12/02/2025    14:43      Program Files (x86)
d-r--          12/02/2025    14:42          Users
d-----          20/02/2026    13:03        Windows

PS C:\> |
```

Il comando cd nel Prompt dei Comandi (CMD)

L'analisi del comando `cd` all'interno del Prompt dei Comandi (CMD) permette di osservare le radici della gestione dei sistemi Windows e le differenze strutturali rispetto a PowerShell. Mentre PowerShell è un ambiente orientato agli oggetti che accetta alias derivati da Unix, il CMD è un interprete legacy più rigido. Come evidenziato dallo screenshot, il tentativo di eseguire `pwd` fallisce poiché il Prompt non riconosce i comandi tipici delle shell moderne, limitandosi esclusivamente alle istruzioni interne del set di comandi Windows tradizionale.

L'attività di navigazione e ispezione nel CMD presenta tratti distintivi rilevanti per la gestione dei rischi e l'analisi di sistema:

- **Assenza di Alias:** La mancata esecuzione di `pwd` dimostra che il CMD non possiede lo strato di astrazione presente in PowerShell. Per un analista, questo significa dover padroneggiare un set di comandi specifico e meno intuitivo per ottenere le medesime informazioni di contesto.
- **Output Testuale Semplificato:** Utilizzando il comando `dir`, il Prompt restituisce un elenco puramente testuale. Sebbene siano presenti dati come data e ora di modifica, l'output manca della colonna "Mode" dettagliata di PowerShell. Qui le directory sono identificate semplicemente dal tag `<DIR>`, offrendo una granularità minore sulla protezione dei file (come i flag di sola lettura o di sistema) a colpo d'occhio.
- **Rigidità della Navigazione:** L'uso di `cd \` per tornare alla root funziona in entrambi gli ambienti, ma nel CMD l'interazione con il file system è meno fluida. Non è possibile, ad

esempio, navigare all'interno del registro di sistema o di altri provider come se fossero directory, limitando l'azione del comando al solo storage fisico.

Dal punto di vista della Cybersecurity, il CMD rimane uno strumento fondamentale per l'analisi "essenziale". Tuttavia, la sua incapacità di trattare i file come oggetti rende più complessa l'estrazione rapida di metadati specifici, richiedendo spesso l'uso di filtri testuali esterni (come `findstr`) per isolare anomalie nel file system durante un audit.

```
C:\Users\develop>pwd
"pwd" non è riconosciuto come comando interno o esterno,
un programma eseguibile o un file batch.

C:\Users\develop>cd Documents

C:\Users\develop\Documents> cd \

C:\>dir
Il volume nell'unità C non ha etichetta.
Numero di serie del volume: B4E6-796E

Directory di C:\

01/04/2024  09:45    <DIR>          PerfLogs
20/02/2026  12:08    <DIR>          Program Files
12/02/2025  14:43    <DIR>          Program Files (x86)
12/02/2025  14:42    <DIR>          Users
20/02/2026  13:03    <DIR>          Windows
             0 File                0 byte
             5 Directory  35.695.263.744 byte disponibili
```

Parte 3: Esplorare i cmdlet

In PowerShell, i comandi non si chiamano semplicemente "comandi", ma **cmdlet** (pronunciato *command-let*). La loro caratteristica principale è la struttura **Verbo-Nome** (ad esempio: *Get-Help*, *Set-Content*, *Stop-Process*). Questa struttura rende il linguaggio estremamente intuitivo: il verbo indica l'azione, il nome indica l'oggetto dell'azione.

In questa fase dell'analisi, viene esaminata la struttura logica dei comandi di PowerShell, che si differenzia radicalmente dal Prompt dei Comandi per l'utilizzo dei **cmdlet**. Un cmdlet è un comando nativo di PowerShell, costruito secondo una convenzione grammaticale fissa: **Verbo-Nome** (es. *Get-ChildItem*). Questa struttura rende il linguaggio estremamente leggibile e coerente.

a. Identificazione dei cmdlet tramite Alias

Sebbene in PowerShell sia possibile utilizzare comandi storici come `dir`, questi non sono veri programmi indipendenti ma **Alias**. Un alias è un soprannome mnemonico che punta al cmdlet reale basato su .NET.

Eseguendo il comando: `Get-Alias dir`

L'output ottenuto conferma che:

- **Nome dell'Alias:** `dir`
- **Comando Reale:** `Get-ChildItem`

Questo meccanismo di astrazione permette agli utenti che provengono da ambienti DOS o Linux (dove si usa `ls`) di operare immediatamente, mentre il motore **PowerShell 7.5.4** traduce silenziosamente l'input nel rispettivo oggetto .NET. Come evidenziato dai test effettuati, l'utilizzo del comando `Get-Command -CommandType Cmdlet | Measure-Object` rivela la presenza di ben **654 cmdlet** disponibili nella sessione attuale, a dimostrazione della vastità del framework rispetto alle poche decine di comandi del CMD.

```
PS C:\Users\develop> Get-Alias dir

CommandType      Name                Version Source
-----
Alias            dir -> Get-ChildItem
```

```
PS C:\Users\develop> Get-Command -CommandType Cmdlet | Measure-Object

Count           : 654
Average         :
Sum             :
Maximum         :
Minimum         :
StandardDeviation :
Property        :
```

```
Administrator: PowerShell 7 (i) x + v
Update-Help

PS C:\Users\develop> Get-Command

CommandType      Name                                     Version      Source
-----
Alias             Add-AppPackage                        2.0.1.0      Appx
Alias             Add-AppPackageVolume                 2.0.1.0      Appx
Alias             Add-AppProvisionedPackage            3.0          Dism
Alias             Add-MsixPackage                      2.0.1.0      Appx
Alias             Add-MsixPackageVolume                2.0.1.0      Appx
Alias             Add-MsixVolume                       2.0.1.0      Appx
Alias             Add-ProvisionedAppPackage            3.0          Dism
Alias             Add-ProvisionedAppSharedPackageContainer 3.0          Dism
Alias             Add-ProvisionedAppxPackage           3.0          Dism
Alias             Add-ProvisioningPackage              3.0          Provisioning
Alias             Add-TrustedProvisioningCertificate    3.0          Provisioning
Alias             Apply-WindowsUnattend               3.0          Dism
Alias             Disable-PhysicalDiskIndication        2.0.0.0      Storage
Alias             Disable-PhysicalDiskIndication        1.0.0.0      VMDirectStorage
Alias             Disable-StorageDiagnosticLog          2.0.0.0      Storage
Alias             Disable-StorageDiagnosticLog          1.0.0.0      VMDirectStorage
Alias             Dismount-AppPackageVolume            2.0.1.0      Appx
Alias             Dismount-MsixPackageVolume           2.0.1.0      Appx
Alias             Dismount-MsixVolume                  2.0.1.0      Appx
Alias             Enable-PhysicalDiskIndication          2.0.0.0      Storage
Alias             Enable-PhysicalDiskIndication          1.0.0.0      VMDirectStorage
Alias             Enable-StorageDiagnosticLog           2.0.0.0      Storage
Alias             Enable-StorageDiagnosticLog           1.0.0.0      VMDirectStorage
```

b. Analisi della documentazione ufficiale

La ricerca effettuata sulla documentazione ufficiale Microsoft (come mostrato negli screenshot) chiarisce la natura dei cmdlet:

- **Definizione:** I cmdlet sono "comandi nativi di PowerShell, non eseguibili autonomi". Essi vengono raccolti in **moduli** che possono essere caricati a seconda delle necessità (sicurezza, rete, gestione cloud).
- **Integrazione .NET:** Ogni cmdlet accetta e restituisce **oggetti .NET**. Questo significa che i dati non fluiscono come semplice testo, ma come strutture ricche di informazioni.
- **Vantaggio Cyber:** Per un analista di cybersecurity, la documentazione rivela che i cmdlet offrono un sistema di aiuto integrato (**Get-Help**) e un sistema di formattazione estensibile, permettendo di estrarre informazioni specifiche (come hash di file o certificati) senza dover "pulire" manualmente l'output testuale.

PowerShell

PanoramicaDSCPowershellGetModuli di utilitàBrowser dei moduliBrowser APIRisorse

Installa PowerShell

Versione

PowerShell 7.5

Trova per titolo

Come utilizzare questa documentazione

Panoramica

Che cos'è PowerShell?

Che cos'è Windows PowerShell?

Che cos'è una shell dei comandi?

Che cos'è un comando PowerShell?

Scopri PowerShell

Installare

Imparare PowerShell

Novità di PowerShell

Panoramica

Novità di PowerShell 7.6

Novità di PowerShell 7.5

Novità di PowerShell 7.4

Novità di PowerShell 7.3

Novità di PowerShell 7.2

Migrazione da Windows PowerShell 5.1 a PowerShell 7

Differenze tra Windows PowerShell 5.1 e PowerShell 7.x

Differenze di PowerShell sulle piattaforme non Windows

Cronologia delle versioni di moduli e cmdlet

Compatibilità del modulo

Windows PowerShell

Sicurezza

Imparare / PowerShell /

Chiedi Impara

Modalità di messa a fuoco

Che cos'è PowerShell?

PowerShell è una soluzione multiplatforma per l'automazione delle attività composta da una shell a riga di comando, un linguaggio di scripting e un framework di gestione della configurazione. PowerShell è compatibile con Windows, Linux e macOS.

Shell della riga di comando

PowerShell è una moderna shell di comandi che include le migliori funzionalità di altre shell popolari. A differenza della maggior parte delle shell che accettano e restituiscono solo testo, PowerShell accetta e restituisce oggetti .NET. La shell include le seguenti funzionalità:

- Cronologia robusta della riga di comando
- Completamento tramite Tab e previsione dei comandi (vedere [about_PSReadLine](#))
- Supporto alias di comandi e parametri
- Pipeline per il concatenamento dei comandi
- Sistema di aiuto nella console , simile alle [man](#) pagine Unix

Linguaggio di scripting

Come linguaggio di scripting, PowerShell è comunemente utilizzato per automatizzare la gestione dei sistemi. Viene anche utilizzato per creare, testare e distribuire soluzioni, spesso in ambienti CI/CD. PowerShell è basato sul Common Language Runtime (CLR) .NET. Tutti gli input e gli output sono oggetti .NET. Non è necessario analizzare l'output di testo per estrarre informazioni dall'output. Il linguaggio di scripting PowerShell include le seguenti funzionalità:

- Estendibile tramite funzioni , classi , script e moduli
- Sistema di formattazione estensibile per un output facile
- Sistema di tipi estensibile per la creazione di tipi dinamici
- Supporto integrato per formati di dati comuni come CSV , JSON e XML

In questo articolo

Shell della riga di comando

Linguaggio di scripting

Piattaforma di automazione

Manifesto della Monade

Prossimi passi

Questa pagina ti è stata utile?

SI

NO

Learn

Documentazione

Formazione e laboratori

Domande e risposte

Argomenti

Accedi

PowerShell

Informazioni generaliDSCPowershellGetModuli di utilitàAPI BrowserRisorse

Installa PowerShell

Parti di questo argomento potrebbero essere state tradotte automaticamente o con l'intelligenza artificiale.

Versione

PowerShell 7.5

Trova in base al titolo

Come usare questa documentazione

Overview

Che cos'è PowerShell?

Che cos'è Windows PowerShell?

Che cos'è una shell dei comandi?

Che cos'è un comando di PowerShell?

Individuare PowerShell

Install

Apprendimento di PowerShell

Novità di PowerShell

Overview

Novità di PowerShell 7.6

Novità di PowerShell 7.5

Novità di PowerShell 7.4

Novità di PowerShell 7.3

Novità di PowerShell 7.2

Migrazione da Windows PowerShell 5.1 a PowerShell 7

Differenze tra Windows PowerShell 5.1 e PowerShell 7.x

Differenze di PowerShell nelle piattaforme non Windows

Cronologia delle versioni di moduli e cmdlet

Compatibilità dei moduli

Scarica il PDF

Learn / PowerShell /

Modalità messa a fuoco

Che cos'è un comando di PowerShell (cmdlet)?

I comandi per PowerShell sono noti come cmdlet (pronunciati command-let). Oltre ai cmdlet, PowerShell consente di eseguire qualsiasi comando disponibile nel sistema.

Che cos'è un cmdlet?

I cmdlet sono comandi nativi di PowerShell, non eseguibili autonomi. I cmdlet vengono raccolti nei moduli di PowerShell che possono essere caricati su richiesta. I cmdlet possono essere scritti in qualsiasi linguaggio .NET compilato o nel linguaggio di scripting di PowerShell stesso.

Nomi dei cmdlet

PowerShell usa una *coppia nome verbo-sostantivo* per denominare i cmdlet. Ad esempio, il `Get-Command` cmdlet incluso in PowerShell viene usato per ottenere tutti i cmdlet registrati nella shell dei comandi. Il verbo identifica l'azione eseguita dal cmdlet e il sostantivo identifica la risorsa in cui il cmdlet esegue l'azione.

Passaggi successivi

Per altre informazioni su PowerShell e su come trovare altri cmdlet, vedere l'esercitazione [Sull'individuazione di PowerShell in Bit di PowerShell](#).

Per altre informazioni sulla creazione di cmdlet personalizzati, vedere le risorse seguenti:

Cmdlet basati su script

- [about_Functions_Advanced](#)

In questo articolo

Che cos'è un cmdlet?

Nomi dei cmdlet

Passaggi successivi

Questa pagina è stata utile?

SI

No

Parte 4: Esplorare il comando netstat usando PowerShell

In questa sezione viene analizzato l'utilizzo di `netstat` (Network Statistics), uno strumento fondamentale per il monitoraggio delle connessioni di rete e la diagnostica della sicurezza. Anche in PowerShell 7.5.4, `netstat` rimane un comando essenziale per visualizzare lo stato dei protocolli e le tabelle di routing.

In ambito **Cybersecurity**, `netstat` è vitale per tre motivi principali:

-Rilevare intrusioni (Malware)

Se un virus o un trojan entra nel tuo computer, quasi certamente cercherà di "chiamare casa" (il server dell'hacker) per inviare i tuoi dati. Usando `netstat`, puoi vedere connessioni verso indirizzi IP sospetti o stranieri che non dovrebbero esserci.

- Monitorare le porte aperte

Ogni porta aperta è un potenziale punto di ingresso per un hacker. Se vedi una porta aperta (stato LISTENING) che non riconosci, potrebbe essere un servizio vulnerabile o un "backdoor" installato da qualcuno.

- Analisi del traffico (Routing)

Con il comando `netstat -r` (quello che hai fatto tu), vedi la **Tabella di Routing**. Questa dice al computer: "Se vuoi andare su Internet, passa da questa porta (Gateway)". Se un attaccante modifica questa tabella, potrebbe dirottare tutto il tuo traffico verso un server malevolo senza che tu te ne accorga.

a. Consultazione della guida (`netstat -h`)

L'esecuzione di `netstat -h` (come mostrato nello screenshot) visualizza l'elenco completo delle opzioni disponibili. È importante notare che, a differenza dei cmdlet nativi, `netstat` è un eseguibile di sistema che restituisce un output testuale non strutturato.

- L'interfaccia di aiuto elenca parametri critici per la cybersecurity, come `-a` (mostra tutte le connessioni e porte in ascolto) e `-o` (mostra l'ID del processo proprietario di ogni connessione), permettendo di identificare potenziali software malevoli che comunicano con l'esterno.

```

PS C:\Users\develop> netstat -h

Mostra le statistiche del protocollo e le connessioni di rete TCP/IP correnti.

NETSTAT [-a] [-b] [-e] [-f] [-i] [-n] [-o] [-p proto] [-r] [-s] [-t] [-x] [-y] [interval]

-a          Mostra tutte le connessioni e le porte di ascolto.
-b          Mostra l'eseguibile coinvolto nella creazione di ogni connessione o
           porta di ascolto. In alcuni casi, eseguibili noti ospitano
           più componenti indipendenti e in questi casi la
           sequenza dei componenti coinvolti nella creazione della connessione
           o della porta di ascolto viene visualizzata. In questo caso, il nome dell'eseguibile
           è in [] in basso, in alto si trova il componente chiamato,
           e così via fino al raggiungimento di TCP/IP. Tenere presente che questa opzione
           può essere dispendiosa in termini di tempo e non andrà a buon fine a meno che non si disponga delle
           autorizzazioni sufficienti.
-c          Visualizza un elenco di processi ordinati in base al numero di
TCP o UDP   porte attualmente utilizzate.
-d          Mostra il valore DSCP associato a ogni connessione.
-e          Mostra le statistiche Ethernet. Potrebbe essere in combinazione con l'opzione
           -s.
-f          Mostra Fully Qualified Domain Names (FQDN) per gli indirizzi
           stranieri.
-i          Mostra il tempo in cui una connessione TCP si trova nel suo stato corrente.
-n          Mostra i numeri di indirizzi e porte in formato numerico.
-o          Mostra l'ID processo di proprietà associato a ogni connessione.
-p proto    Mostra le connessioni per il protocollo specificato dal protocollo; il protocollo
           può essere: TCP, UDP, TCPv6 o UDPv6. Se usato con l'opzione -s
           per mostrare le statistiche per protocollo, il protocollo potrebbe essere:
           IP, IPv6, ICMP, ICMPv6, TCP, TCPv6, UDP o UDPv6.
-q          Mostra tutte le connessioni, le porte di ascolto e le porte
           TCP non di ascolto associate. Le porte non di ascolto associate potrebbero essere associate o meno
           a una connessione attiva.
-r          Mostra la tabella di routing.
-s          Mostra le statistiche per protocollo. Per impostazione predefinita, le statistiche vengono
           mostrate per IP, IPv6, ICMP, ICMPv6, TCP, TCPv6, UDP e UDPv6;
           l'opzione -p potrebbe essere usata per specificare un sottoinsieme dell'opzione predefinita.
-t          Mostra lo stato di offload della connessione corrente.
-x          Mostra connessioni NetworkDirect, listener ed endpoint
           condivisi.
-y          Mostra il modello di connessione TCP per tutte le connessioni.
           Non può essere in combinazione con altre opzioni.
interval    Mostra di nuovo le statistiche selezionate, inserendo intervalli di secondi
           tra ogni visualizzazione. Premi CTRL+C per interrompere la nuova visualizzazione delle
           statistiche. Se omesso, netstat stamperà le

```

b. Analisi della Tabella di Routing (netstat -r)

L'output di `netstat -r` fornisce una panoramica dettagliata di come il sistema instrada il traffico dati. Analizzando lo screenshot della **Tabella route IPv4**, emergono dati cruciali per l'analisi della rete:

- **Gateway Predefinito:** L'indirizzo `192.168.64.1` è il punto di uscita della rete locale verso internet.
- **Interfaccia Locale:** L'indirizzo `192.168.64.23` identifica univocamente la macchina virtuale dell'analista (Red Hat VirtIO Ethernet Adapter).
- **Rotte Attive:** La presenza di rotte come `0.0.0.0` (la rotta di default) indica che il sistema è configurato correttamente per comunicare con reti esterne.

```

PS C:\Users\develop> netstat -r
=====
Elenco interfacce
 5...56 f2 5e 61 ff 2f .....Red Hat VirtIO Ethernet Adapter
 1.....Software Loopback Interface 1
=====

IPv4 Tabella route
=====
Route attive:
  Indirizzo rete      Mask      Gateway      Interfaccia  Metrica
 0.0.0.0              0.0.0.0    192.168.64.1  192.168.64.23  15
 127.0.0.0            255.0.0.0    On-link      127.0.0.1     331
 127.0.0.1            255.255.255.255  On-link      127.0.0.1     331
 127.255.255.255      255.255.255.255  On-link      127.0.0.1     331
 192.168.64.0         255.255.255.0   On-link      192.168.64.23  271
 192.168.64.23        255.255.255.255  On-link      192.168.64.23  271
 192.168.64.255       255.255.255.255  On-link      192.168.64.23  271
 224.0.0.0            240.0.0.0    On-link      127.0.0.1     331
 224.0.0.0            240.0.0.0    On-link      192.168.64.23  271
 255.255.255.255      255.255.255.255  On-link      127.0.0.1     331
 255.255.255.255      255.255.255.255  On-link      192.168.64.23  271
=====
Route permanenti:
 Nessuna

IPv6 Tabella route
=====
Route attive:
 Interf Metrica Rete Destinazione      Gateway
 5      271  ::/0              fe80::10bd:3aff:fea6:64
 1      331  ::1/128            On-link
 5      271  fda7:9880:7399:3122::/64 On-link
 5      271  fda7:9880:7399:3122:c4bd:5b49:c1b:80ae/128
                               On-link
 5      271  fda7:9880:7399:3122:d243:1039:bcf0:1d28/128
                               On-link
 5      271  fe80::/64          On-link
 5      271  fe80::d0f3:5456:4607:a6c4/128
                               On-link
 1      331  ff00::/8           On-link
 5      271  ff00::/8           On-link
=====
Route permanenti:
 Nessuna

```

I 3 stati negli screenshot:

- **LISTENING:** Il computer è "in ascolto", come un orecchio teso. Aspetta che qualcuno lo chiami su quella porta.
- **ESTABLISHED:** C'è una conversazione attiva in corso. Tra me e il server remoto, stiamo scambiando dati.
- **CLOSE_WAIT / TIME_WAIT:** La conversazione è finita e il sistema sta "pulendo" la linea prima di chiuderla del tutto.

Differenza Tecnica tra CMD e PowerShell 7.5.4

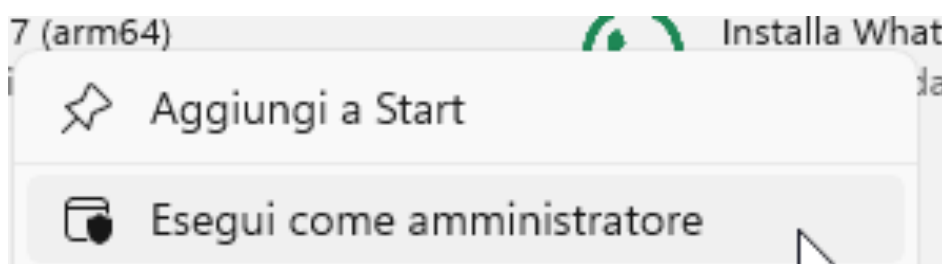
Sebbene l'output visivo di `netstat` sia identico in entrambi gli ambienti, PowerShell 7.5.4 permette di gestire queste informazioni con una logica superiore. Mentre nel Prompt dei Comandi i dati di `netstat` sono solo testo da leggere, in PowerShell è possibile utilizzare cmdlet alternativi come `Get-NetRoute`.

A differenza del vecchio comando, `Get-NetRoute` estrae le stesse informazioni dello screenshot ma le trasforma in **oggetti .NET**. Questo permette, ad esempio, di creare script automatici che bloccano istantaneamente il traffico se rilevano un Gateway non autorizzato, un'operazione che richiederebbe ore di lavoro manuale se fatta tramite il semplice testo del Prompt.

In questa fase avanzata, l'obiettivo è identificare non solo quali connessioni sono attive, ma quale specifico software le stia generando. Questo processo di **correlazione tra rete e sistema operativo** è un pilastro della *Threat Hunting*.

c. Esecuzione con privilegi elevati

Per ottenere la piena visibilità sulle connessioni, è stata utilizzata una sessione di **PowerShell 7.5.4 con privilegi di Amministratore**. L'elevazione dei privilegi è necessaria affinché il comando possa accedere alle tabelle dei descrittori dei processi di sistema e associare ogni porta al relativo eseguibile.



d. Il comando `netstat -abno`

L'esecuzione del comando `netstat -abno` ha permesso di estrarre un set di dati completo:

- **-a:** Visualizzazione di tutte le connessioni e porte in ascolto.
- **-b:** Identificazione dell'eseguibile coinvolto (fondamentale per distinguere processi legittimi da potenziali malware).
- **-n:** Visualizzazione degli indirizzi in formato numerico per evitare ritardi dovuti alla risoluzione DNS.
- **-o:** Associazione di ogni riga a un **PID (Process ID)** univoco.

Dall'analisi dell'output, è stata isolata una connessione **ESTABLISHED** (attiva) verso l'IP esterno `20.50.88.234:443`, associata all'eseguibile `[ pwsh.exe ]` con il **PID 2904**.

Route permanenti:

Nessuna

PS C:\Users\develop> (netstat -abno)

Connessioni attive

Proto	Indirizzo locale	Indirizzo esterno	Stato	PID
TCP	0.0.0.0:135	0.0.0.0:0	LISTENING	552
RpcSs				
[svchost.exe]				
TCP	0.0.0.0:445	0.0.0.0:0	LISTENING	4
Impossibile ottenere informazioni sulla proprietà				
TCP	0.0.0.0:5040	0.0.0.0:0	LISTENING	4972
CDPSvc				
[svchost.exe]				
TCP	0.0.0.0:7680	0.0.0.0:0	LISTENING	7912
Impossibile ottenere informazioni sulla proprietà				
TCP	0.0.0.0:49664	0.0.0.0:0	LISTENING	836
Impossibile ottenere informazioni sulla proprietà				
TCP	0.0.0.0:49665	0.0.0.0:0	LISTENING	672
Impossibile ottenere informazioni sulla proprietà				
TCP	0.0.0.0:49666	0.0.0.0:0	LISTENING	1496
Schedule				
[svchost.exe]				
TCP	0.0.0.0:49667	0.0.0.0:0	LISTENING	1932
EventLog				
[svchost.exe]				
TCP	0.0.0.0:49668	0.0.0.0:0	LISTENING	3096
[spoolsv.exe]				
TCP	0.0.0.0:49670	0.0.0.0:0	LISTENING	816
Impossibile ottenere informazioni sulla proprietà				
TCP	127.0.0.1:9843	0.0.0.0:0	LISTENING	3456
[spice-webdavd.exe]				
TCP	192.168.64.23:139	0.0.0.0:0	LISTENING	4
Impossibile ottenere informazioni sulla proprietà				
TCP	192.168.64.23:50924	52.110.19.224:443	ESTABLISHED	9980
[StartMenuExperienceHost.exe]				

[svchost.exe]				
TCP	0.0.0.0:49667	0.0.0.0:0	LISTENING	1932
EventLog				
[svchost.exe]				
TCP	0.0.0.0:49668	0.0.0.0:0	LISTENING	3096
[spoolsv.exe]				
TCP	0.0.0.0:49670	0.0.0.0:0	LISTENING	816
Impossibile ottenere informazioni sulla proprietà				
TCP	127.0.0.1:9843	0.0.0.0:0	LISTENING	3456
[spice-webdavd.exe]				
TCP	192.168.64.23:139	0.0.0.0:0	LISTENING	4
Impossibile ottenere informazioni sulla proprietà				
TCP	192.168.64.23:50924	52.110.19.224:443	ESTABLISHED	9980
[StartMenuExperienceHost.exe]				
TCP	192.168.64.23:60836	4.207.247.139:443	ESTABLISHED	3552
WpnService				
[svchost.exe]				
TCP	192.168.64.23:61368	20.50.88.234:443	ESTABLISHED	2904
[pwsh.exe]				
TCP	192.168.64.23:62600	92.124.106.123:443	CLOSE_WAIT	8412
[SearchHost.exe]				
TCP	192.168.64.23:62602	13.107.219.254:443	CLOSE_WAIT	8412
[SearchHost.exe]				
TCP	192.168.64.23:62606	72.153.5.139:443	ESTABLISHED	7912
Impossibile ottenere informazioni sulla proprietà				
TCP	192.168.64.23:62607	72.153.5.139:443	ESTABLISHED	7912
Impossibile ottenere informazioni sulla proprietà				
TCP	192.168.64.23:62608	72.153.5.139:443	ESTABLISHED	7912
Impossibile ottenere informazioni sulla proprietà				
TCP	192.168.64.23:62609	72.153.5.133:443	ESTABLISHED	7912

e, f, g. Analisi tramite Gestione Attività (Task Manager)

Utilizzando la scheda **Dettagli** del Task Manager e ordinando i processi per PID, è stato individuato il corrispondente processo **pwsh.exe** (PID 2904). L'analisi della finestra **Proprietà** dell'eseguibile ha fornito informazioni cruciali:

- **Percorso del file:** L'eseguibile si trova in **C:\Program Files\PowerShell\7\pwsh.exe**. La posizione in una cartella di sistema protetta conferma la legittimità del software.
- **Dettagli Versione:** Viene confermata l'esecuzione di PowerShell 7, basata sul runtime moderno .NET.
- **Firme Digitali:** La scheda Proprietà permette di verificare che il file sia firmato digitalmente da Microsoft. In un contesto di analisi forense, l'assenza di una firma valida per un processo che comunica in rete (come il PID 2904 analizzato) rappresenterebbe un indicatore di compromissione (IoC).

Informazioni ottenibili dalla scheda Dettagli e Proprietà:

Dalla scheda Dettagli e dalla finestra di dialogo Proprietà per il PID selezionato, è possibile ottenere:

1. **Architettura:** (es. Arm64), fondamentale per confermare l'ottimizzazione per l'hardware specifico in uso.
2. **Stato del processo:** (In esecuzione/Sospeso).
3. **Nome Utente:** Il contesto di sicurezza sotto cui gira il processo (nel tuo caso, l'utente `develop`).
4. **Descrizione e Dimensioni:** Informazioni fisiche sul file per escludere tecniche di *masquerading* (es. un file chiamato **pwsh.exe** ma con dimensioni o descrizioni errate).

Dettagli

Nome	PID	Stato	Nome ute...	CPU	Memoria (...)	Archite...	Descrizione
dwim.exe	1072	In esecuzione	DWM-1	00	64.084 K	Arm64	Gestione finestre desktop
svchost.exe	1120	In esecuzione	SERVIZIO ...	00	1.696 K	Arm64	Processo host per servizi di Windows
svchost.exe	1212	In esecuzione	SYSTEM	00	952 K	Arm64	Processo host per servizi di Windows
svchost.exe	1232	In esecuzione	SERVIZIO ...	00	3.404 K	Arm64	Processo host per servizi di Windows
svchost.exe	1364	In esecuzione	SERVIZIO L...	00	308 K	Arm64	Processo host per servizi di Windows
svchost.exe	1380	In esecuzione	SYSTEM	00	664 K	Arm64	Processo host per servizi di Windows
svchost.exe	1392	In esecuzione	SERVIZIO L...	00	1.164 K	Arm64	Processo host per servizi di Windows
svchost.exe	1496	In esecuzione	SYSTEM	00	3.776 K	Arm64	Processo host per servizi di Windows
svchost.exe	1504	In esecuzione	SERVIZIO L...	00	1.520 K	Arm64	Processo host per servizi di Windows
svchost.exe	1516	In esecuzione	SERVIZIO L...	00	888 K	Arm64	Processo host per servizi di Windows
svchost.exe	1524	In esecuzione	SERVIZIO L...	00	1.884 K	Arm64	Processo host per servizi di Windows
svchost.exe	1540	In esecuzione	SYSTEM	00	636 K	Arm64	Processo host per servizi di Windows
ShellExperienceHost...	1588	Sospeso	develop	00	0 K	Arm64	Windows Shell Experience Host
svchost.exe	1632	In esecuzione	SYSTEM	00	1.492 K	Arm64	Processo host per servizi di Windows
svchost.exe	1664	In esecuzione	SYSTEM	00	1.116 K	Arm64	Processo host per servizi di Windows
svchost.exe	1672	In esecuzione	SERVIZIO ...	00	2.312 K	Arm64	Processo host per servizi di Windows
OneDrive.Sync.Servi...	1712	In esecuzione	develop	00	4.752 K	Arm64	Microsoft OneDrive Sync Service
svchost.exe	1724	In esecuzione	SYSTEM	00	1.128 K	Arm64	Processo host per servizi di Windows
svchost.exe	1824	In esecuzione	SERVIZIO L...	00	1.084 K	Arm64	Processo host per servizi di Windows
svchost.exe	1932	In esecuzione	SERVIZIO L...	00	9.772 K	Arm64	Processo host per servizi di Windows
XtaCache.exe	1944	In esecuzione	SYSTEM	00	392 K	Arm64	XTA Cache Service
svchost.exe	2012	In esecuzione	SERVIZIO L...	00	740 K	Arm64	Processo host per servizi di Windows
svchost.exe	2052	In esecuzione	SYSTEM	00	424 K	Arm64	Processo host per servizi di Windows
svchost.exe	2316	In esecuzione	SYSTEM	00	504 K	Arm64	Processo host per servizi di Windows
svchost.exe	2436	In esecuzione	SYSTEM	00	624 K	Arm64	Processo host per servizi di Windows
svchost.exe	2460	In esecuzione	SERVIZIO L...	00	928 K	Arm64	Processo host per servizi di Windows
RuntimeBroker.exe	2468	In esecuzione	develop	00	988 K	Arm64	Runtime Broker
svchost.exe	2580	In esecuzione	develop	00	1.780 K	Arm64	Processo host per servizi di Windows
svchost.exe	2640	In esecuzione	SERVIZIO L...	00	820 K	Arm64	Processo host per servizi di Windows
svchost.exe	2656	In esecuzione	SERVIZIO L...	00	992 K	Arm64	Processo host per servizi di Windows
svchost.exe	2684	In esecuzione	SERVIZIO L...	00	1.648 K	Arm64	Processo host per servizi di Windows
svchost.exe	2696	In esecuzione	develop	00	1.044 K	Arm64	Processo host per servizi di Windows
svchost.exe	2768	In esecuzione	SYSTEM	00	452 K	Arm64	Processo host per servizi di Windows
svchost.exe	2828	In esecuzione	SERVIZIO L...	00	948 K	Arm64	Processo host per servizi di Windows
svchost.exe	2852	In esecuzione	SERVIZIO L...	00	568 K	Arm64	Processo host per servizi di Windows
pwsh.exe	2904	In esecuzione	develop	00	36.428 K	Arm64	PowerShell 7
Widgets.exe	2964	In esecuzione	develop	00	7.676 K	Arm64	Widgets
svchost.exe	3000	In esecuzione	SERVIZIO L...	00	872 K	Arm64	Processo host per servizi di Windows
conhost.exe	3048	In esecuzione	develop	00	432 K	Arm64	Host finestra console
svchost.exe	3060	In esecuzione	SYSTEM	00	12.256 K	Arm64	Processo host per servizi di Windows
spoolsv.exe	3096	In esecuzione	SYSTEM	00	1.012 K	Arm64	Applicazione sottosistema spooler
svchost.exe	3168	In esecuzione	SERVIZIO L...	00	7.712 K	Arm64	Processo host per servizi di Windows
svchost.exe	3200	In esecuzione	SERVIZIO ...	00	680 K	Arm64	Processo host per servizi di Windows
svchost.exe	3236	In esecuzione	SYSTEM	00	3.908 K	Arm64	Processo host per servizi di Windows
svchost.exe	3268	In esecuzione	SYSTEM	00	624 K	Arm64	Processo host per servizi di Windows
svchost.exe	3352	In esecuzione	SYSTEM	00	9.248 K	Arm64	Processo host per servizi di Windows
svchost.exe	3360	In esecuzione	SERVIZIO L...	00	21.120 K	Arm64	Processo host per servizi di Windows

Proprietà - pwsh

Sicurezza

Dettagli

Versioni precedenti

Generale

Compatibilità

Firme digitali

pwsh

Tipo di file: Applicazione (.exe)

Descrizione: PowerShell 7

Percorso: C:\Program Files\PowerShell\7

Dimensioni: 270 KB (276.512 byte)

Dimensioni su disco: 272 KB (278.528 byte)

Data creazione: giovedì 16 ottobre 2025, 03:00:12

Ultima modifica: giovedì 16 ottobre 2025, 03:00:12

Ultimo accesso: Oggi 20 febbraio 2026, 2 minuti fa

Attributi: ☐ Solo lettura ☐ Nascosto

Avanzate...

OK

Annulla

Applica

Parte 5: Svuotare il cestino usando PowerShell

L'utilizzo del comando `Clear-RecycleBin` esemplifica come un'operazione che normalmente richiederebbe l'uso dell'interfaccia grafica (individuare l'icona, cliccare col tasto destro, confermare) possa essere ridotta a una singola riga di comando, facilmente inseribile in uno script di manutenzione automatizzata.

a, b, c. Esecuzione del comando

Dopo aver verificato la presenza di file nel Cestino (come file di testo o documenti di test), è stato eseguito il cmdlet: `Clear-RecycleBin`

Il sistema ha richiesto una conferma (`Confirm`), una misura di sicurezza standard di PowerShell per le azioni irreversibili. Digitando `y` (Yes), il comando è stato completato.

Cosa è successo ai file nel Cestino? I file sono stati eliminati permanentemente dal file system. A differenza dell'eliminazione semplice, questa operazione libera lo spazio su disco e rimuove i riferimenti logici ai file dal contenitore di sistema del Cestino. Per un analista, questo comando è utile per garantire che dati sensibili analizzati durante una sessione di lavoro non rimangano in aree di stoccaggio temporaneo del sistema operativo.

```
PS C:\Users\develop> Clear-RecycleBin
```

Confirm

Are you sure you want to perform this action?

Performing the operation "Clear-RecycleBin" on target "All of the contents of the Recycle Bin".

[Y] Yes [A] Yes to All [N] No [L] No to All [S] Suspend [?] Help (default is "Y"): Y

```
PS C:\Users\develop> |
```

